

Mini-projet — Application Web Flask et gestion de projet avec GitHub

Objectif du projet

Vous allez réaliser, en groupe de **2 à 3 étudiants**, une petite application Web avec **Python, Flask, HTML/Jinja et CSS**.

Vous utiliserez également **Git, GitHub et GitHub Projects** pour organiser et suivre votre travail.

L'objectif n'est pas de créer une application complexe.

Vous devez montrer que vous êtes capables de :

- comprendre un besoin ;
- découper un projet en tâches ;
- répartir le travail ;
- planifier les tâches ;
- développer une application Flask simple ;
- travailler avec Git et GitHub ;
- tester et valider une application ;
- documenter votre travail ;
- analyser le déroulement réel du projet.

Application attendue : fonctionnelle, simple, claire, testée et documentée.

Compétence travaillées

C03 — Gérer un projet

Les activités/tâches

- **R4 – Gestion de projet et d'équipe**
 - **R4-T1** — Identification des étapes du projet —  ;
 - **R4-T2** — Identification des ressources humaines et matérielles —  ;
 - **R4-T5** — Pilotage de l'exécution des travaux — .
- organiser le travail ;
- identifier les tâches ;
- répartir les ressources ;
- planifier le projet ;
- suivre l'avancement ;
- analyser les écarts.

C08 — Coder

Les activités/tâches

- **D2 – Développement et validation de solutions logicielles**
 - **D2-T1** — Conception de l'architecture d'une solution logicielle —  ;
 - **D2-T3** — Développement, utilisation ou adaptation de composants logiciels — .
- comprendre une architecture Flask ;
- adapter du code Python ;
- développer des routes ;
- utiliser les templates Jinja ;
- exploiter les données fournies ;
- développer une fonctionnalité simple.

C06 — Valider un système informatique

Les activités/tâches

- **D2 – Développement et validation de solutions logicielles**
 - **D2-T4** — Tests de mise en production —  ;
 - **D2-T5** — Recette et validation — .
- tester l'application ;
- vérifier les fonctionnalités ;
- identifier les anomalies ;
- corriger les problèmes ;
- réaliser une validation finale.

1. Le projet

Votre groupe devra :

1. choisir un sujet ;
2. prendre en main le squelette Flask fourni ;
3. créer et organiser le dépôt GitHub ;
4. créer les Issues et le GitHub Project ;
5. répartir les tâches ;
6. réaliser un Gantt prévisionnel ;

7. développer l'application ;
 8. utiliser des branches, commits et Pull Requests ;
 9. tester l'application ;
 10. réaliser la recette ;
 11. rédiger le README ;
 12. comparer le planning prévu avec le déroulement réel ;
 13. réaliser un bilan collectif.
-

2. Choisir un sujet

Chaque groupe choisit **un seul sujet**.

Sujet 1 — Gestion du matériel informatique

Une entreprise souhaite consulter l'inventaire de son parc informatique.

Informations possibles :

- catégorie ;
- marque ;
- statut ;
- localisation ;
- description.

Sujet 2 — Suivi des incidents informatiques

Un service informatique souhaite consulter les incidents signalés par les utilisateurs.

Informations possibles :

- priorité ;
- statut ;
- catégorie ;
- utilisateur concerné ;
- description.

Sujet 3 — Inventaire des équipements réseau

Une entreprise souhaite consulter les équipements de son infrastructure réseau.

Informations possibles :

- adresse IP ;
- type ;
- localisation ;
- statut ;
- description.

Sujet 4 — Suivi des interventions techniques

Une entreprise souhaite suivre les interventions réalisées ou planifiées.

Informations possibles :

- technicien ;
- client ;
- type d'intervention ;
- statut ;
- description.

Sujet 5 — Suivi de la maintenance des équipements

Un service informatique souhaite suivre les opérations de maintenance.

Informations possibles :

- équipement ;
- technicien ;
- type de maintenance ;
- statut ;
- description.

Les données correspondant à votre sujet vous seront fournies dans un fichier JSON.

3. Fonctionnalités attendues

Tous les sujets reposent sur le même fonctionnement :

```
JSON
↓
Python / Flask
↓
Jinja
↓
HTML
↓
CSS
↓
Navigateur
```

Votre application doit proposer au minimum :

Page liste

Elle doit permettre de :

- afficher les éléments du fichier JSON ;
- présenter clairement leurs informations principales ;
- accéder à la page de détail ;
- filtrer les éléments.

Page détail

Elle doit afficher les informations détaillées d'un élément sélectionné.

Exemple :

PC-001

Catégorie : Ordinateur

Marque : Dell

Statut : En service

Localisation : Bureau 101

Description :

Ordinateur portable utilisé par le service administratif.

[Retour à la liste]

Filtre

Vous devez développer **au moins un filtre fonctionnel**.

Exemples :

Statut : [Tous ▼]

[Filtrer]

ou :

Priorité : [Toutes ▼]

Le filtre doit réellement modifier les résultats affichés.

Le cas sans résultat doit également être géré :

Aucun résultat correspondant à votre recherche.

4. Interface

Une première feuille de style CSS vous sera fournie.

Vous devez la personnaliser.

Votre interface doit présenter :

- des titres clairement hiérarchisés ;
- une mise en page lisible ;
- des couleurs cohérentes ;
- des blocs ou cartes pour les informations ;
- des liens ou boutons clairement identifiables ;
- une présentation adaptée aux statuts.

Cherchez la simplicité plutôt que la complexité.

Votre interface doit être :

simple, propre, claire et lisible.

5. Prendre en main Flask

Un squelette Flask fonctionnel vous sera fourni.

```
mini-projet-flask/  
├── app.py  
├── data/  
│   └── data.json  
├── templates/  
│   ├── base.html  
│   ├── index.html  
│   └── detail.html  
├── static/  
│   └── style.css  
├── tests/  
│   └── fiche_tests.md  
├── requirements.txt  
├── .gitignore  
└── README.md
```

Vous devez comprendre

Élément	Rôle
app.py	Application Flask et routes

Élément	Rôle
<code>data.json</code>	Données
<code>templates/</code>	Pages HTML/Jinja
<code>static/</code>	CSS et ressources statiques
<code>tests/</code>	Tests
<code>requirements.txt</code>	Dépendances
<code>README.md</code>	Documentation

Vous n'avez pas besoin de connaître tout Flask.

Vous devez comprendre suffisamment le squelette pour pouvoir l'adapter.

Première vérification

Installez les dépendances :

```
pip install -r requirements.txt
```

Lancez l'application :

```
python app.py
```

Ouvrez ensuite l'adresse indiquée dans le terminal, généralement :

```
http://127.0.0.1:5000
```

Vérifiez que :

- ☐ la page liste fonctionne ;
- ☐ la page détail fonctionne ;
- ☐ les données sont affichées.

Ne commencez pas les modifications avant d'avoir vérifié que le squelette fonctionne.

6. Organiser le dépôt GitHub

Un membre du groupe crée un dépôt GitHub public.

Nom :

```
mini-projet-flask-groupeX
```

Ajoutez ensuite les autres membres comme collaborateurs.

Dans le dépôt GitHub, utilisez :

```
Settings → Collaborators
```

ou, selon l'interface :

```
Settings → Collaborators and teams → Add people
```

Ajoutez chaque membre à partir de son **nom d'utilisateur GitHub**.

Chaque étudiant doit accepter l'invitation reçue.

Chaque étudiant doit pouvoir :

- ☐ accéder au dépôt ;
- ☐ récupérer le projet avec `git clone` ou avec le téléchargement ZIP ;
- ☐ lancer l'application ;
- ☐ créer une branche ;
- ☐ faire un commit ;
- ☐ envoyer ses modifications avec `git push` ;
- ☐ récupérer les modifications du groupe avec `git pull`.

Premier envoi

Après avoir récupéré le squelette :

```
git init
git status
git add .
git commit -m "Initialisation du projet Flask"
git remote add origin URL_DU_DEPOT
git push -u origin main
```

Chaque étudiant doit ensuite cloner le dépôt avec Git (ou télécharger le dépôt au format ZIP).

7. Organiser le travail avec GitHub

Issues

Une **Issue** = une tâche.

Exemples :

```
Adapter les données JSON
Créer la page liste
Créer la page détail
Ajouter le filtre
Améliorer le CSS
Tester la page liste
Rédiger le README
Effectuer la recette
```

Évitez :

```
Faire le site
```

Préférez :

```
Créer la page détail des équipements
```

Chaque Issue doit comporter :

- un titre explicite ;
- une description ;
- un responsable ;
- un label.

Labels recommandés

Label	Utilisation
<code>feature</code>	Fonctionnalité
<code>interface</code>	HTML / CSS
<code>test</code>	Test / validation
<code>documentation</code>	Documentation
<code>bug</code>	Correction
<code>project</code>	Gestion du projet

GitHub Project

Créez un **GitHub Project** associé au dépôt.

Projects → New project

Utilisez une vue **Board** avec au minimum :

Backlog → À faire → En cours → Terminé

Colonne	Signification
Backlog	Tâches identifiées mais pas encore planifiées
À faire	Tâches prévues et prêtes à être commencées
En cours	Tâches actuellement réalisées
Terminé	Tâches réalisées, testées, vérifiées et intégrées

Une tâche passe dans **Terminé** uniquement lorsqu'elle est :

- développée ;
- testée ;
- vérifiée ;
- intégrée au projet.

Le tableau doit être mis à jour pendant toute la durée du projet.

8. Planifier le projet

Avant le développement, identifiez les principales tâches.

Pour chaque tâche, indiquez :

- responsable ;
- début ;
- fin ;
- durée ;
- dépendances éventuelles.

Exemple :

Tâche	Responsable	Début	Fin
Prise en main Flask	A	J1	J1
Affichage	B	J1	J2
Page détail	A	J2	J2

Tâche	Responsable	Début	Fin
Filtre	B	J2	J3
CSS	C	J2	J3
Tests	B	J3	J4
Documentation	Groupe	J4	J4

Gantt

À partir de ce planning, réalisez un **diagramme de Gantt**.

Le Gantt représente :

ce qui était prévu.

Il doit présenter :

- les tâches ;
- les responsables ;
- les dates ;
- la durée ;
- les dépendances pertinentes.

Roadmap

Utilisez la vue **Roadmap** de GitHub Projects pour suivre le déroulement réel.

Retenez :

Gantt = prévision
Roadmap = suivi réel

Vous comparerez les deux à la fin du projet.

9. Développer avec Git

Chaque tâche doit être réalisée sur une branche identifiable.

Le cycle attendu est :

```
Issue
↓
Branche
↓
Modification
↓
Commit
```

```
↓  
Push  
↓  
Pull Request  
↓  
Review  
↓  
Merge
```

Créer une branche

```
git switch main  
git pull origin main  
git switch -c feature-page-liste
```

Exemples :

```
feature-page-detail  
feature-filtre-statut  
interface-css  
test-page-liste  
docs-readme
```

Commit

```
git status  
git add .  
git commit -m "Ajout du filtre par statut"  
git push -u origin feature-filtre-statut
```

Les messages doivent être explicites.

Exemples :

```
Ajout du filtre par statut  
Création de la page détail  
Correction de l'affichage du statut
```

Évitez :

```
modif  
test
```

ça marche
version finale

10. Pull Request et vérification

Lorsqu'une tâche est terminée, créez une **Pull Request** vers **main**.

Un autre membre du groupe doit vérifier le travail.

Il doit notamment vérifier :

- ☐ l'application fonctionne ;
- ☐ la fonctionnalité correspond à l'Issue ;
- ☐ aucune fonctionnalité existante n'est cassée ;
- ☐ le code reste compréhensible ;
- ☐ l'interface reste cohérente ;
- ☐ les tests nécessaires ont été réalisés.

Si nécessaire, corrigez le travail avant le merge.

Les modifications ne doivent pas être envoyées directement sur **main.**

11. Tester et valider

Avant de considérer l'application comme terminée, vous devez la tester.

Tests à réaliser

Affichage

- ☐ la page principale fonctionne ;
- ☐ les données sont correctement affichées ;
- ☐ la page détail fonctionne.

Filtre

- ☐ le filtre donne un résultat ;
- ☐ le filtre donne plusieurs résultats ;
- ☐ le filtre peut donner zéro résultat ;
- ☐ le message « aucun résultat » fonctionne.

Navigation

- ☐ les liens fonctionnent ;
- ☐ l'accès au détail fonctionne ;
- ☐ le retour vers la liste fonctionne.

Interface

- ☐ les informations sont lisibles ;
- ☐ les statuts sont correctement présentés ;
- ☐ la mise en page est cohérente.

12. Fiche de tests et recette

Créez :

```
tests/fiche_tests.md
```

Exemple :

N°	Test	Résultat attendu	Résultat obtenu	Statut
T01	Ouvrir /	Liste affichée	Liste affichée	OK
T02	Ouvrir un détail	Détail affiché	Détail affiché	OK
T03	Filtrer	Résultats filtrés	Résultats filtrés	OK
T04	Aucun résultat	Message affiché	Message affiché	OK
T05	Retour liste	Liste affichée	Liste affichée	OK

Recette

Vérifiez ensuite que le produit final correspond au cahier des charges.

Fonctionnalité	Conforme ?
Liste	Oui / Non
Détail	Oui / Non
Filtre	Oui / Non
Cas sans résultat	Oui / Non
Navigation	Oui / Non
CSS	Oui / Non

Une fonctionnalité non conforme doit être corrigée lorsque cela est possible.

13. Documenter le projet

Votre **README.md** doit présenter :

Présentation

Quel est le rôle de votre application ?

Fonctionnalités

Qu'avez-vous développé ?

Installation

```
pip install -r requirements.txt
```

Lancement

```
python app.py
```

Organisation

Présentez les principaux fichiers et dossiers.

Membres

Indiquez les membres du groupe et leur rôle principal.

Gestion du projet

Présentez :

- les Issues ;
- le GitHub Project ;
- le Kanban ;
- le planning ;
- les principales Pull Requests.

Bilan

Présentez :

- les difficultés rencontrées ;
- les solutions trouvées ;
- les écarts entre prévision et réalisation ;
- les améliorations possibles.

14. Comparer prévision et réalisation

À la fin du projet, comparez :

Gantt prévisionnel ↔ Roadmap GitHub

Répondez aux questions suivantes :

- Le planning initial a-t-il été respecté ?
- Quelles tâches ont pris plus de temps ?
- Quelles tâches ont été plus rapides ?
- Des tâches ont-elles été ajoutées ?
- Des tâches ont-elles été supprimées ou modifiées ?
- Pourquoi ?
- Le Gantt était-il réaliste ?
- Qu'avez-vous appris sur la planification ?

15. Livrables

Dans le dépôt GitHub

```
mini-projet-flask/  
├── app.py  
├── data/  
│   └── data.json  
├── templates/  
│   ├── base.html  
│   ├── index.html  
│   └── detail.html  
├── static/  
│   └── style.css  
├── tests/  
│   └── fiche_tests.md  
├── requirements.txt  
├── .gitignore  
└── README.md
```

Le dépôt doit également présenter :

- ☐ les Issues ;
- ☐ les labels ;
- ☐ les branches ;
- ☐ les commits ;
- ☐ les Pull Requests ;
- ☐ le GitHub Project ;

- ☐ la Roadmap.

À fournir

- ☐ le Gantt prévisionnel ;
 - ☐ le dépôt GitHub complet ;
 - ☐ la fiche de tests ;
 - ☐ le README ;
 - ☐ le bilan du projet.
-

16. Checklist finale

Avant de rendre le projet, vérifiez :

Application

- ☐ Flask se lance correctement.
- ☐ Les données JSON sont correctement chargées.
- ☐ La page liste fonctionne.
- ☐ La page détail fonctionne.
- ☐ Le filtre fonctionne.
- ☐ Le cas sans résultat est géré.
- ☐ Le CSS a été personnalisé.
- ☐ L'interface est claire et lisible.

Tests

- ☐ Les tests ont été réalisés.
- ☐ La fiche de tests est complétée.
- ☐ La recette a été effectuée.
- ☐ Les problèmes identifiés ont été corrigés lorsque cela était possible.

Git / GitHub

- ☐ Chaque membre a contribué.
- ☐ Les Issues sont utilisées.
- ☐ Les branches sont utilisées.
- ☐ Les commits sont explicites.
- ☐ Les Pull Requests sont utilisées.
- ☐ Les revues ont été réalisées.
- ☐ Le GitHub Project est à jour.

Projet

- ☐ Le Gantt est fourni.
- ☐ La Roadmap est renseignée.
- ☐ La comparaison Gantt / Roadmap est réalisée.

- ☐ Le README est complet.
 - ☐ Le bilan collectif est réalisé.
-

17. Bilan collectif

Répondez collectivement aux questions suivantes :

1. Quelle était votre principale difficulté technique ?
 2. Quelle était votre principale difficulté dans le travail en groupe ?
 3. Quelle tâche a demandé le plus de temps ?
 4. Votre planning était-il réaliste ?
 5. Quels écarts observez-vous entre le Gantt et la Roadmap ?
 6. Quel est l'intérêt des Issues ?
 7. Quel est l'intérêt des branches ?
 8. Quel est l'intérêt de GitHub Projects ?
 9. Qu'amélioreriez-vous si vous deviez refaire le projet ?
 10. Quelle fonctionnalité ajouteriez-vous dans une version 2 ?
-

18. Consigne finale

Votre objectif n'est pas de produire une application complexe.

Vous devez montrer que vous êtes capables de **mener un petit projet informatique de manière organisée**, depuis l'analyse du besoin jusqu'à la validation de la solution.

Votre projet doit être :

fonctionnel · simple · clair · testé · documenté

Votre dépôt GitHub doit également permettre de comprendre :

comment le travail a été organisé, réalisé et validé.

Lorsque toutes les cases de la checklist finale sont cochées, le projet peut être considéré comme terminé.
